



Mobile and Autonomous Robots

UE23CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu

TA : Gunda Sukesh

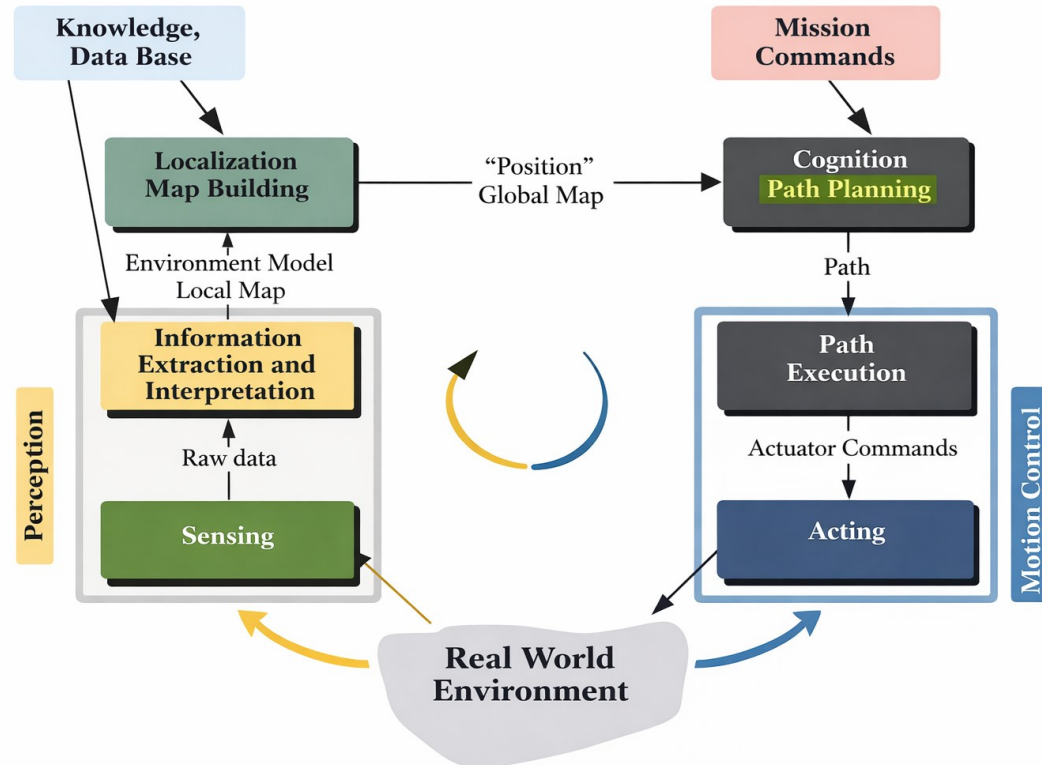


Mobile and Autonomous Robots

UE23CS343BB7

Unit- 4: Navigation

Introduction Path Planning



Robotic Cognition and Navigation

- Navigation represents the "cognitive" level of a mobile robot, moving beyond basic movement to purposeful decision-making.
- It is the ability to reach a goal reliably using a mix of prior knowledge (maps) and real-time sensor data.
- While many navigation architectures exist, they all fundamentally focus on how to break down the complex problem of "getting from A to B" into smaller, executable subunits.

Strategic Planning vs. Tactical Reaction

- Robust navigation relies on two distinct extremes: path planning and obstacle avoidance.
- Path planning is the strategic, long-term competence of identifying a trajectory based on a map to reach a destination.
- In contrast, obstacle avoidance is a tactical, real-time necessity that modulates that trajectory to prevent immediate collisions.
- Balancing these two allows a robot to remain goal-oriented while staying safe in dynamic environments.

- While often viewed as opposites, planning and reacting are complementary essentials in robotics.
- Planning provides the long-term strategy to reach a distant goal, while reacting allows the robot to handle unforeseen obstacles and dynamic changes.
- Without a plan, a robot has no direction; without reaction, it cannot survive the physical journey.
- The ideal "integrated" system merges these two, instantly updating its strategic plan as new sensor information arrives.

- Because a robot can never be 100% certain of its exact physical location, it navigates based on a belief state.
- A plan is essentially a trajectory designed to move the robot from its “initial” belief to a "goal" belief.
- A critical measure of these systems is completeness: a system is complete if it is guaranteed to find a solution whenever one exists.
- In practice, designers often sacrifice completeness for faster computation, making it vital to understand where and why a system might fail.

- Path planning was initially extensively studied for industrial manipulator robotics due to its applications.
- Manipulators with multiple degrees of freedom present more complex path-planning challenges compared to differential-drive robots.
- Path-planning algorithms for mobile robots are often simpler due to reduced degrees of freedom.
- Industrial robots prioritize speed for high throughput, considering both kinematics and dynamics in path planning.
- Mobile robots, often operating at lower speeds, typically focus on kinematics rather than dynamics in path planning, simplifying the problem

Mobile and Autonomous Robots

Path Planning

Path Planning Strategies

1. Graph Search

- A connectivity graph in free space is first constructed and then searched.
- The graph construction process is often performed offline.

2. Potential Field Planning

- A mathematical function is imposed directly on the free space.
- The gradient of this function can then be followed to the goal.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu



Mobile and Autonomous Robots

UE23CS343BB7

Navigation

Graph Search and Potential Field Path Planning

- Graph search techniques have traditionally been strongly rooted in the field of mathematics.
- Recent innovation in the robotics community has been largely driven by the necessity for real-time algorithms that can accommodate evolving maps and dynamic graphs.
- For most of these methods we distinguish two main steps:
 - **Graph Construction:** Here nodes are placed and connected via edges.
 - **Graph Search:** Here the computation of an (optimal) solution is performed.

Mobile and Autonomous Robots

Graph Search: Visibility Graphs

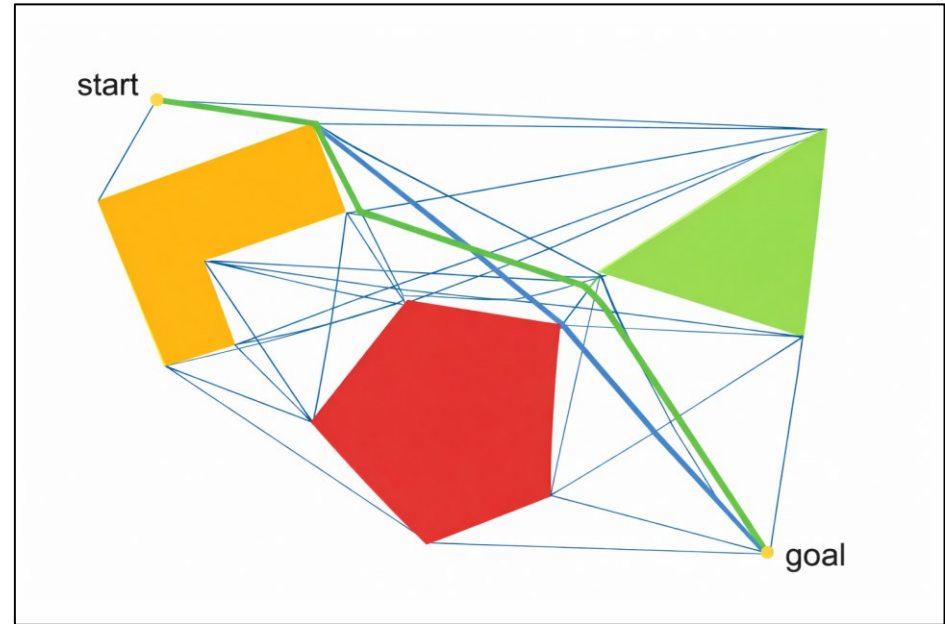
- The visibility graph for a polygonal configuration space consists of edges joining all pairs of vertices that can see each other (including both the initial and goal positions as vertices as well).
- The unobstructed straight lines (roads) joining those vertices are obviously the shortest distances between them.
- The task of the path planner is then to find a (shortest) path from the initial position to the goal position along the roads defined by the visibility graph.
- Visibility graphs are widely used in mobile robotics due to their simple implementation and their ability to directly utilise polygonal obstacle descriptions to define a roadmap of mutually visible vertices.

- Two primary caveats exist when employing visibility graph search: the method becomes computationally inefficient in dense environments as the number of nodes and edges scales with obstacle count, and the resulting paths prioritize distance over safety by grazing obstacles.
- While these paths are mathematically optimal in length, they pose a significant collision risk by bringing the robot as close to obstacles as possible.
- To address this safety concern, developers typically inflate obstacle boundaries or adjust the path afterward to create a buffer.
- However, implementing these safety measures inevitably sacrifices the original shortest-path optimality.

Mobile and Autonomous Robots

Graph Search: Visibility Graphs

The nodes of the graph are the initial and goal points and the vertices of the configuration space obstacles (polygons). All nodes which are visible from each other are connected by straight-line segments, defining the road map. This means there are also edges along each polygon's sides.



Mobile and Autonomous Robots

Graph Search: Voronoi Diagram

- A Voronoi diagram is a complete road map method that tends to maximize the distance between the robot and obstacles in the map.
- For each point in free space, its distance to the nearest obstacle is computed. If you plot that distance as the height coming out of the page, it increases as you move away from an obstacle .
- At points that are equidistant from two or more obstacles, such a distance plot has sharp ridges.
- The Voronoi diagram consists of the edges formed by these sharp ridge points.
- When the configuration space obstacles are polygons, the Voronoi diagram consists of straight line and parabolic segments only.

Mobile and Autonomous Robots

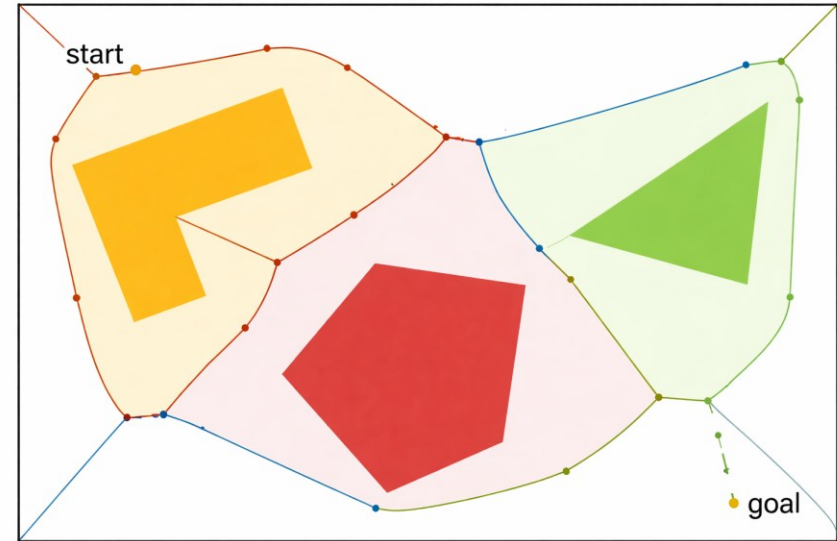
Graph Search: Voronoi Diagram

- While Voronoi road maps and visibility graphs both guarantee completeness, Voronoi-based solutions are typically much longer and far from optimal in terms of total path length.
- Voronoi diagrams pose challenges for robots with limited range sensors, as their edges maximize distance to obstacles, potentially leading to poor localization and navigation.
- Despite this drawback, Voronoi diagram-based paths offer executability, as robots equipped with range sensors can efficiently follow Voronoi edges using simple control rules, aiding in navigation and map construction.

Mobile and Autonomous Robots

Graph Search: Voronoi Diagram

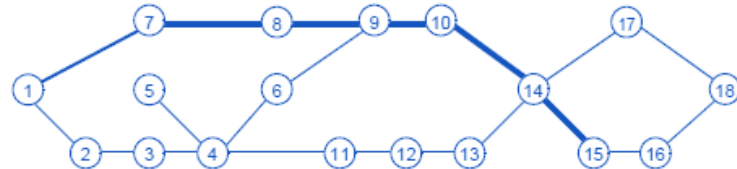
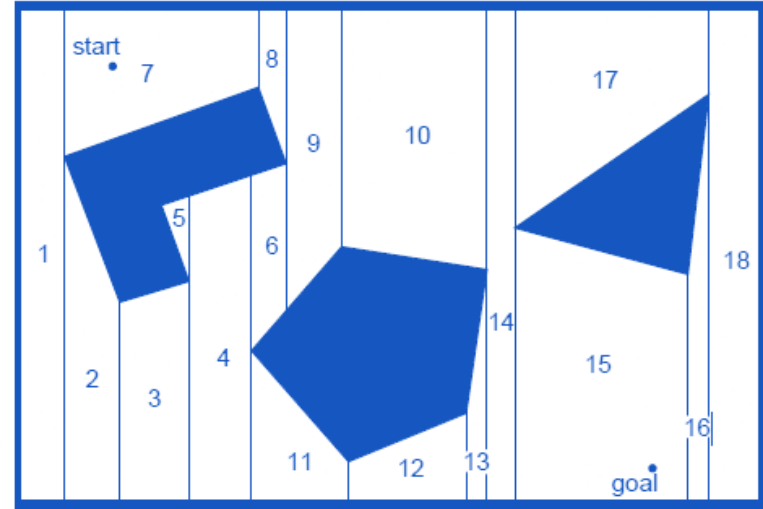
The Voronoi diagram consists of the lines constructed from all points that are equidistant from two or more obstacles. The initial q_{init} and q_{goal} goal configurations are mapped into the Voronoi diagram to q'_{init} and q'_{goal} , each by drawing the line along which its distance to the boundary of the obstacles increases the fastest. The points on the Voronoi diagram represent transitions from straight line segments (minimum distance between two lines) to parabolic segments (minimum distance between a line and a point).



- Exact cell decomposition divides the environment into distinct cells based on geometric criticality, ensuring that every resulting cell is either entirely free or entirely occupied.
- This approach ensures completeness, much like road-map methods, because it focuses on the robot's ability to traverse between adjacent free cells rather than its precise coordinates within them.
- The computational efficiency of this technique depends heavily on the density of objects, making it exceptionally fast in large, sparse environments where fewer cells are required.
- However, because the number of cells increases with environmental complexity, the method can become inefficient when dealing with a high number of obstacles.

- Despite being a robust and lossless representation, its implementation is particularly difficult, which leads to it being used relatively rarely in practical mobile robotics applications.

Here, cells are formed according to the horizontal coordinate of extreme obstacle points.

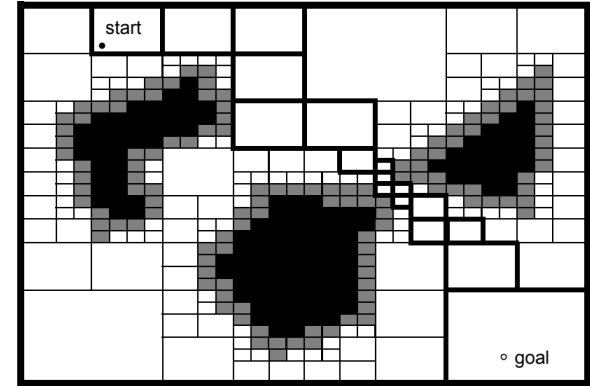
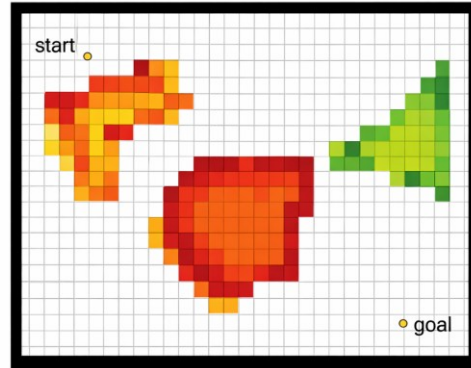
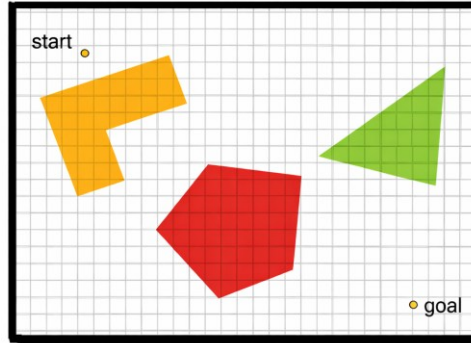


Graph Search: Approximate Cell Decomposition

- Approximate cell decomposition, particularly **fixed-size** and **variable-size** methods, is widely used in mobile robotics, especially with grid-based environmental representations.
- Fixed-size cell decomposition uses a uniform grid where each cell has a predetermined size, which may lead to narrow passages being overlooked due to the grid's inexactness, although this is usually mitigated by using small cell sizes.
- Variable-size approximate cell decomposition employs a hierarchical approach, recursively decomposing the free space into smaller rectangles. Path planning can proceed hierarchically, starting with coarse resolutions and refining until a solution is found or a limit resolution is reached.
- The key advantage of approximate cell decomposition is its low computational complexity, making it a popular choice for path planning in mobile robotics.

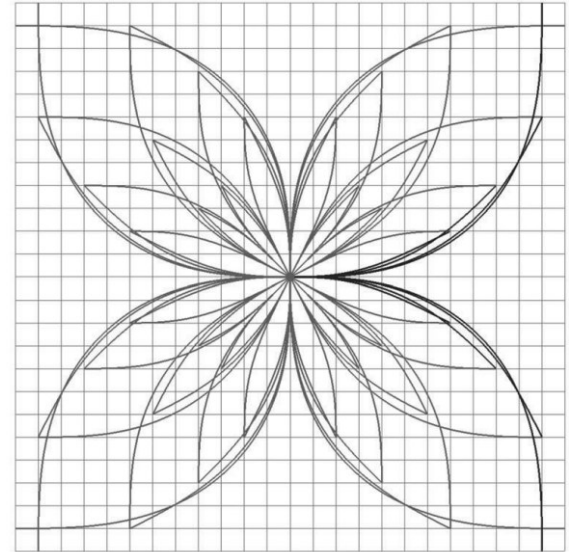
Graph Search: Approximate Cell Decomposition

The bounding rectangle of the free space is subdivided into four identical quadrants, which are then recursively decomposed only if they contain both free space and obstacles. This process continues until a cell is either entirely empty or entirely occupied, or a maximum resolution is reached. In this representation, white cells are free, black cells are occupied, and gray cells indicate a mix of both regions.



- Lattice structures, a recent addition to graph search techniques, involve constructing a base set of edges and replicating it across the configuration space to form a graph, providing design flexibility for feasible edges.
- These structures, akin to approximate cell decomposition, offer executable edges that can be directly implemented by robotic platforms, enhancing their applicability.
- Bicchi et al. applied input discretization to create a configuration space lattice for specific kinematic vehicle models, while Pivtoraiko et al. precomputed lattice graphs by fixing a configuration space discretization and dimensionality, followed by edge pruning for efficiency.
- Lattice graphs are typically precomputed and stored in memory, making them part of the approximate decomposition methods and enabling direct use of edges along the solution path as feed-forward commands for controllers

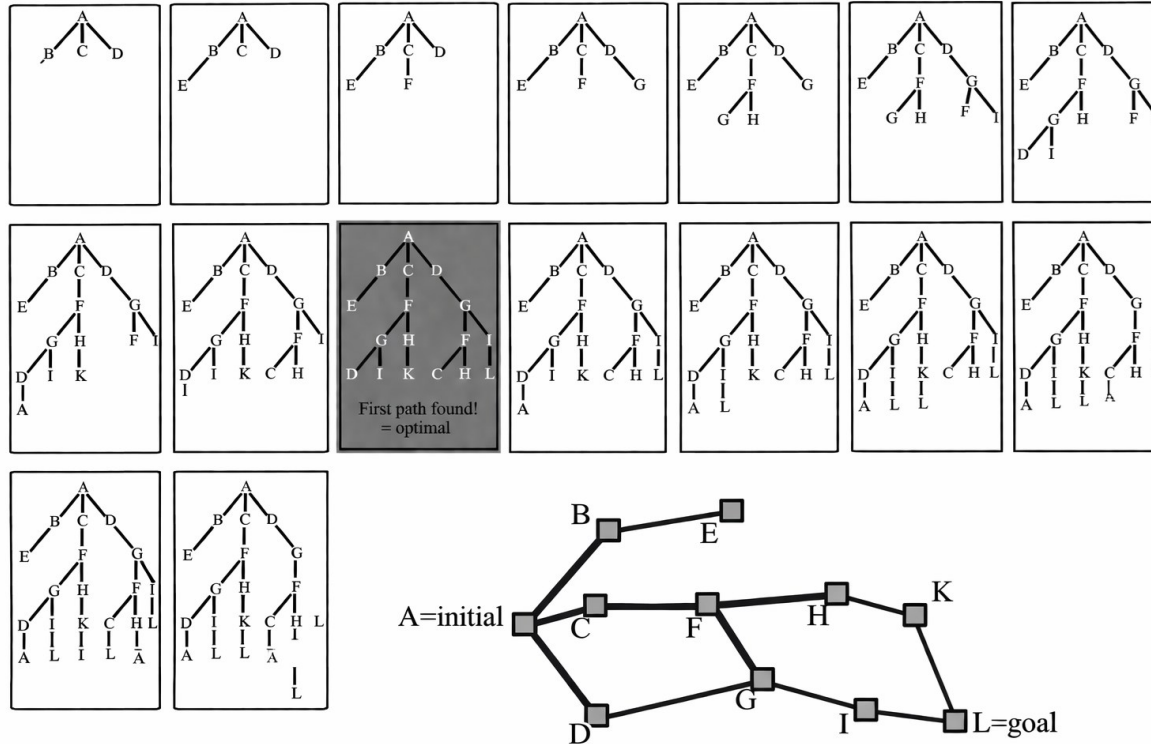
16-directional state lattice constructed for a planetary exploration rover. The state includes 2D position, heading, and curvature (x, y, ϑ, k) . Note that straight segments are part of the lattice set but occluded by longer curved segments. The lattice is 2D-shift invariant and partially invariant to rotation. All successor edges of state $(0, 0, 0, 0)$ are depicted in black.



- Path planning involves finding the optimal path in a connectivity graph derived from the environment map.
- Various search algorithms are employed to determine the best path between the start and goal nodes in the graph.
- Popular search algorithms in mobile robotics include Dijkstra's algorithm, A* algorithm, and probabilistic roadmap algorithms.

- Due to the similarity between many graph search algorithms, we begin with an elaboration on their respective differences.
- It is beneficent to introduce the concepts of expected total cost $f(n)$, path cost $g(n)$, edge traversal cost $c(n, n')$ and heuristic cost $h(n)$ which are all functions of the node n .
- We denote the accumulated cost from the start node to any given node n with $g(n)$.
- The cost from a node n to an adjacent node n' becomes $c(n, n')$, and the expected cost (heuristic cost) from a node n to the goal node is described with $h(n)$. The total expected cost from start to goal via state n can then be written as $f(n) = g(n) + \epsilon \cdot h(n)$, where ϵ is a parameter that assumes algorithm-dependent values.

- In cases where all edges have uniform traversal costs, simpler and faster implementations like depth-first and breadth-first searches are viable.
- However, for nonuniform cost maps, Dijkstra's algorithm and its variants enable the computation of optimal paths albeit with higher algorithmic complexity.
- In all if these implementations, $\epsilon = 0$. In case of $\epsilon \neq 0$, a heuristic function $h(n)$ is employed, which essentially incorporates additional information about the problem set and thus often allows for faster convergence of the search query.



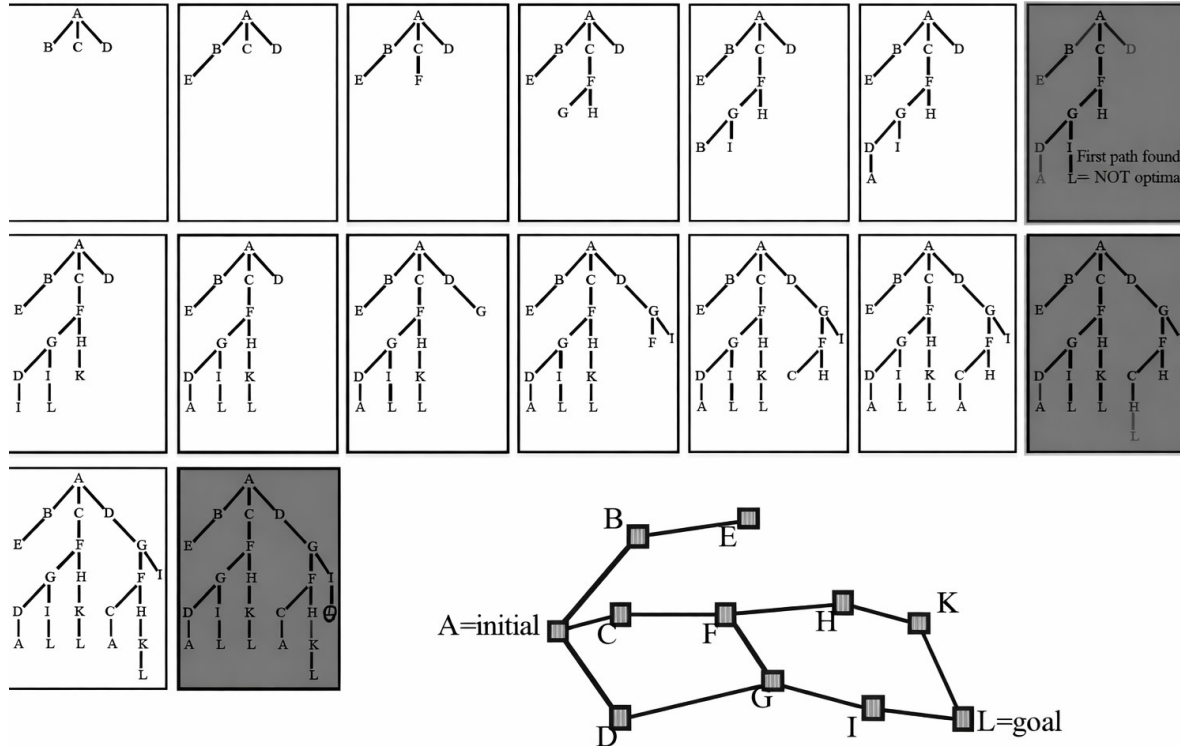
Deterministic Graph Search: Breadth First Search

- Breadth-first search (BFS) explores neighboring nodes starting from the initial node and expands them in order of proximity to the start node, defined by the shortest number of edge transitions.
- BFS terminates upon reaching the goal node, ensuring fast computation as nodes are already sorted by proximity to the start.
- If all edges have constant costs, BFS returns the path with the fewest edges, making it optimal. However, for nonuniform cost graphs, BFS may not find the lowest-cost path.
- In robotics, BFS is applied in algorithms like the wavefront expansion, efficient for finding routes in fixed-size cell arrays by marking each cell's distance to the goal cell and expanding outward from the goal position. It offers linear complexity regardless of environmental sparsity or shape complexity.

Deterministic Graph Search: Breadth First Search

- The time complexity of BFS is $O(V + E)$, where V is the number of vertices (nodes) and E is the number of edges in the graph.
- In the worst case, BFS visits every vertex and every edge once, resulting in a linear time complexity.
- The space complexity of BFS is also $O(V + E)$.
- In the worst case, BFS requires space to store all vertices in the queue and maintain the visited status of each vertex. This results in a linear space complexity.

Deterministic Graph Search: Depth First Search



Deterministic Graph Search: Depth First Search

- In contrast to breadth-first search, depth-first search expands each node up to the deepest level of the graph (until the node has no more successors).
- As those nodes are expanded, their branch is removed from the graph and the search backtracks by expanding the next neighboring node of the start node until its deepest level and so on.
- An inconvenience of this algorithm is that it may revisit previously visited nodes or enter redundant paths.
- However, these situations may be easily avoided through an efficient implementation. A significant advantage of depth-first over breadth-first is space complexity.

Deterministic Graph Search: Depth First Search

- In fact, depth-first needs to store only a single path from the start node to the goal node along with all the remaining unexpanded neighboring nodes for each node on the path. Once each node has been expanded and all its children nodes have been explored, it can be removed from memory.
- The time complexity of DFS is $O(V+E)$ because, in the worst case, every vertex and edge is visited exactly once.
- Similarly, the space complexity is $O(V)$, as the algorithm requires memory for the recursion stack or an explicit stack to track the current path.
- In a worst-case scenario, such as a single long path, the stack must store all vertices, resulting in linear space complexity.

Mobile and Autonomous Robots

Deterministic Graph Search

Some other algorithms which are generally used are:

- Dijkstra Algorithm
- A* Algorithm
- D* Algorithm

Mobile and Autonomous Robots

Randomised Graph Search

- When encountering complex high-dimensional path planning problems (such as in manipulation tasks on robotic arms, or molecule folding and docking queries for drug placement, and so on) it becomes infeasible to solve them exhaustively within reasonable time limits.
- Reverting to heuristic search methods is often not possible due to the lack of an appropriate heuristic function and a reduction of the problem dimensionality frequently fails due to velocity and acceleration constraints imposed on the model, which should not be violated for security reasons.
- In such situations, randomized search becomes useful, since it forgoes solution optimality for faster solution computation.

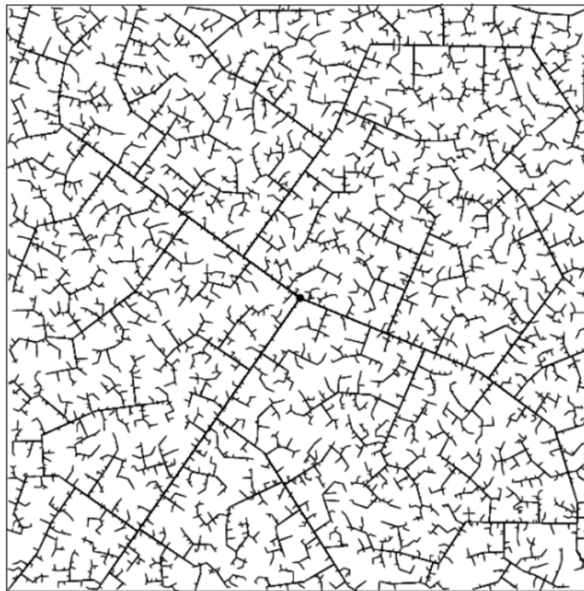
Randomised Graph Search: Rapidly Exploring Random Trees

- RRTs grow a search tree online by iteratively picking a random free-space configuration and extending the nearest existing node toward it using a fixed-length, collision-free edge.
- While the basic algorithm can be slow, efficiency is often improved through bidirectional searches or by biasing random selection toward the goal node.
- Although RRTs lack optimality and deterministic completeness, they are probabilistically complete, ensuring a solution will eventually be found if one exists.

- Evolution of an RRT is shown below.



45 iterations



2345 iterations

- Potential field path planning creates a field, or gradient, across the robot's map that directs the robot to the goal position from multiple prior positions.
- This approach was originally invented for robot manipulator path planning and is used often and under many variants in the mobile robotics community.
- The potential field method treats the robot as a point under the influence of an artificial potential field.
- The robot moves by follow- grand grand qnear qnear grand qnew grand grand $U(q)$. The robot moves by following the fiels, just as a ball would roll downhill.
- The goal (a minimum in this space) acts as an attractive force on the robot, and the obstacles act as peaks, or repulsive forces.

- The superposition of all forces is applied to the robot, which, in most cases, is assumed to be a point in the configuration space.
- Such an artificial potential field smoothly guides the robot toward the goal while simultaneously avoiding known obstacles.
- The basic idea behind all potential field approaches is that the robot is attracted toward the goal, while being repulsed by the obstacles that are known in advance.
- If new obstacles appear during robot motion, one could update the potential field in order to integrate this new information.

Mobile and Autonomous Robots

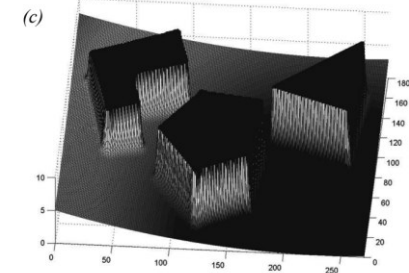
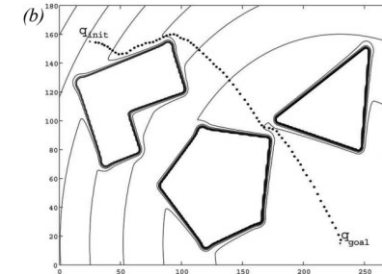
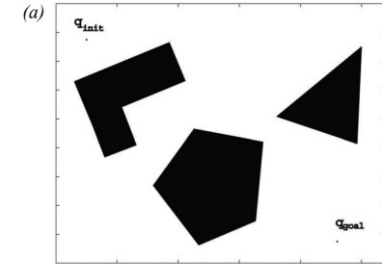
Potential Field Path Planning

Typical potential field generated by the attracting goal and two obstacles.

(a) Configuration of the obstacles, start (top left) and goal (bottom right).

(b) Equipotential plot and path generated by the field.

(c) Resulting potential field generated by the goal attractor and obstacles.





THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu



Mobile and Autonomous Robots

UE23CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu

TA : Gunda Sukesh



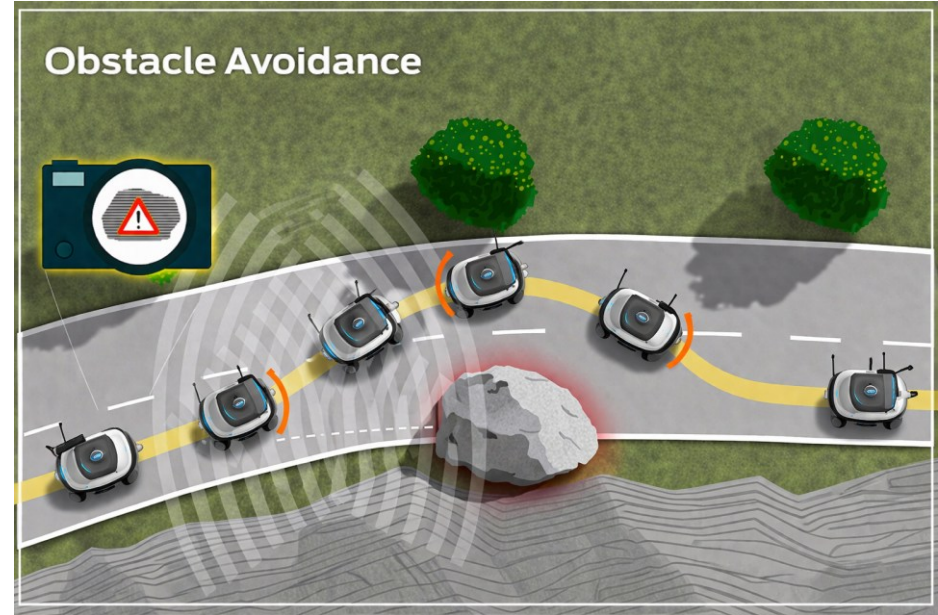
Mobile and Autonomous Robots

UE23CS343BB7

Navigation

**Obstacle Avoidance: Bug algorithm, Vector field
Histogram, Bubble Band Technique**

- Local obstacle avoidance focuses on changing the robot's trajectory as informed by its sensors during robot motion.
- The resulting robot motion is both a function of the robot's current or recent sensor readings and its goal position and relative location to the goal position.



- The Bug algorithm represents a technique in that only the most recent robot sensor values are used, and the robot needs, in addition to current sensor values, only approximate information regarding the direction of the goal.
- The basic idea is to follow the contour of each obstacle in the robot's way and thus circumnavigate it.
- With Bug1, the robot fully circles the object first, then departs from the point with the shortest distance toward the goal (figure 1). This approach is, of course, very inefficient but it guarantees that the robot will reach any reachable goal.

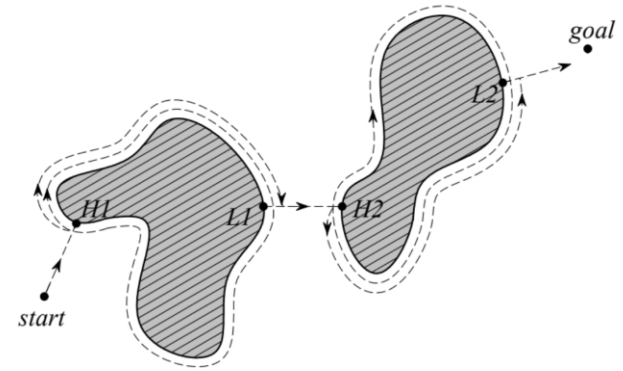


Figure 1: Bug1 algorithm with H1, H2, hit points, and L1, L2, leave points.

- With Bug2 the robot begins to follow the object's contour, but departs immediately when it is able to move directly toward the goal.
- In general this improved Bug algorithm will have significantly shorter total robot travel, as shown in figure 2. However, one can still construct situations in which Bug2 is arbitrarily inefficient (i.e., nonoptimal).
- A number of variations and extensions of the Bug algorithm exist. We mention one more, the *Tangent Bug*, which adds range sensing and a local environmental representation termed the local tangent graph (LTG).

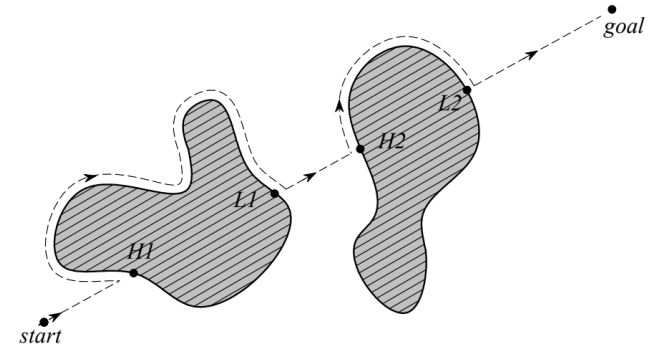


Figure 2: Bug2 algorithm with $H1$, $H2$, hit points, and $L1$, $L2$, leave points.

- The LTG enables more efficient navigation towards the goal and facilitates the discovery of shortcuts around obstacles, allowing the robot to revert to goal-seeking sooner.
- In simpler environments, Tangent Bug methods often yield globally optimal paths.

Practical Application: Example of Bug 2

It outlines the robot's motion in terms of two states: `GOAL-SEEK` for moving toward the goal and `WALLFOLLOW` for moving around obstacles. The approach relies on describing the robot's motion as a function of sensor values and relative direction to the goal for each state. By determining when to switch between these states, a practical implementation of Bug2 for obstacle avoidance can be achieved.

The following pseudocode provides the highest-level control code for such a decomposition.

```
public void bug2(position goalPos) {
    boolean atGoal = false;

    while( ! atGoal){
        position robotPos = robot.GetPos(&sonars);
        distance goalDist = getDistance(robotPos, goalPos);
        angle goalAngle = Math.atan2(goalPos, robotPos)-robot.GetAngle();
        velocity forwardVel, rotationVel;

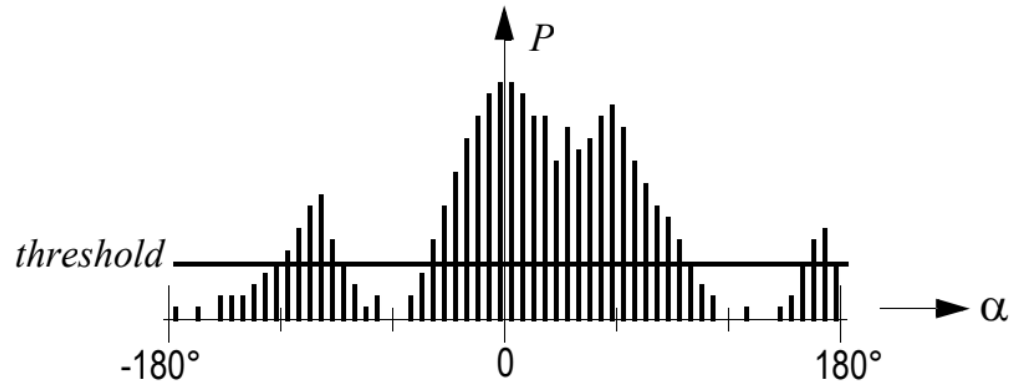
        if(goalDist < atGoalThreshold){
            System.out.println("At Goal!");
            forwardVel = 0;
            rotationVel = 0;
            robot.SetState(DONE);
            atGoal = true;
        }
        else{
            forwardVel = ComputeTranslation(&sonars);
            if(robot.GetState() == GOALSEEK){
                rotationVel = ComputeGoalSeekRot(goalAngle);
                if(ObstaclesInWay(goalAngle, &sonars))
                    robot.SetState(WALLFOLLOW);
            }
            if(robot.GetState() == WALLFOLLOW){
                rotationVel = ComputeRWFRot(&sonars);
                if( ! ObstaclesInWay(goalAngle, &sonars))
                    robot.SetState(GOALSEEK);
            }
        }
        robot.SetVelocity(forwardVel, rotationVel);
    }
}
```


Obstacle Avoidance: Vector Field Histogram

- One of the central criticisms of Bug-type algorithms is that the robot's behavior at each instant is generally a function of only its most recent sensor readings.
- This can lead to undesirable and yet preventable problems in cases where the robot's instantaneous sensor readings do not provide enough information for robust obstacle avoidance.
- The VFH techniques overcome this limitation by creating a local map of the environment around the robot.
- This local map is a small occupancy grid, populated only by relatively recent sensor range readings.

Obstacle Avoidance: Vector Field Histogram

- For obstacle avoidance, VFH generates a polar histogram. The x-axis represents the angle at which the obstacle was found, and the y-axis represents the probability that there really is an obstacle in that direction based on the occupancy grid cell values.



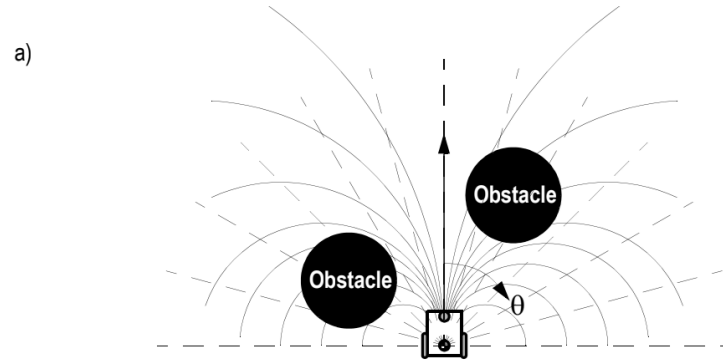
Obstacle Avoidance: Vector Field Histogram

- From this histogram a steering direction is calculated. First, all openings large enough for the vehicle to pass through are identified.
- Then a cost function is applied to every such candidate opening. The passage with the lowest cost is chosen. The cost function has three terms:

$$G = a.\text{target_direction} + b.\text{wheel_orientation} + c.\text{previous_direction}$$

- ❑ target_direction = alignment of the robot path with the goal;
- ❑ wheel_orientation = difference between the new direction and the current wheel orientation;
- ❑ previous_direction = difference between the previously selected direction and the new direction.

- The cost function in obstacle avoidance algorithms like VFH+ is designed to penalize large deviations from the goal direction.
- Parameters such as α and β tune the behavior of the robot, with a higher value for α indicating a stronger bias towards the goal.
- In VFH+ improvement, a reduction stage considers a simplified model of the robot's possible trajectories based on its kinematic limitations, such as turning radius for an Ackerman vehicle.
- The robot's movement is modeled in arcs or straight lines, and an obstacle obstructs all allowable trajectories passing through it. This results in a modified polar histogram where obstacles are enlarged to account for all kinematically blocked trajectories effectively

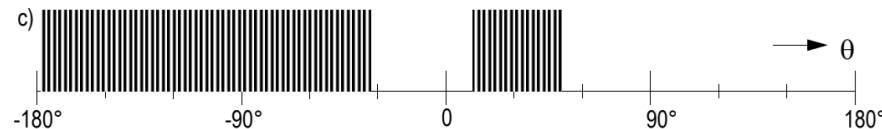


Example of blocked directions and resulting polar histograms.

(a) Robot and blocking obstacles.

(b) Polar histogram.

(c) Masked polar histogram.



Obstacle Avoidance: Bubble Band Technique

- A bubble is defined as the maximum local subset of the free space around a given configuration of the robot that which can be traveled in any direction without collision.
- The bubble is generated using a simplified model of the robot in conjunction with range information available in the robot's map.
- Despite the simplified model, the actual shape of the robot is taken into account to ensure smooth trajectory planning during path execution.
- Given such bubbles, a band or string of bubbles can be used along the trajectory from the robot's initial position to its goal position to show the robot's expected free space throughout its path.

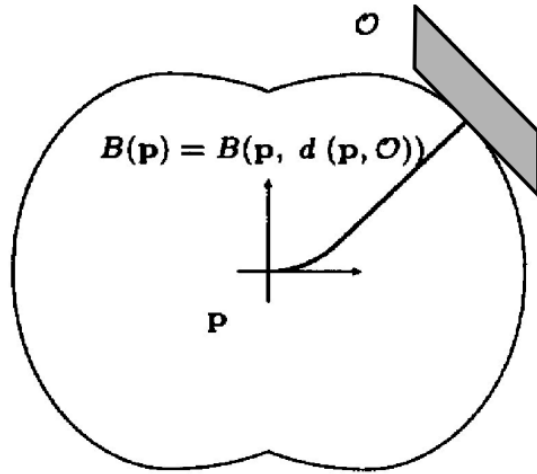


Figure 3: Shape of the bubbles around the vehicle.

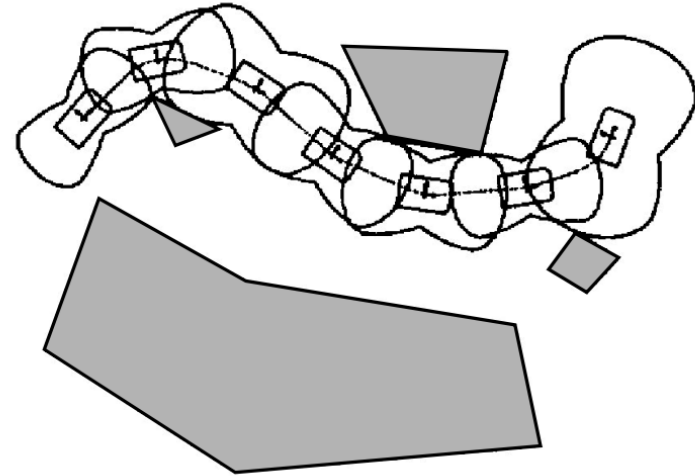


Figure 4: A typical bubble band.

Obstacle Avoidance: Bubble Band Technique

- Initially, this approach leans towards path optimization rather than obstacle avoidance.
- During robot motion, the bubble band strategy helps in obstacle avoidance by deflecting the robot from its planned path based on unforeseen sensor values.
- The aim is to minimise tension within the bubble band, ensuring smooth navigation.
- The technique allows for consideration of the robot's actual dimensions, but it is most effective when the environment configuration is well-known in advance, similar to offline path-planning methods.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu



Mobile and Autonomous Robots

UE23CS343BB7

Navigation

**Obstacle Avoidance: Curvature Velocity
Techniques, Dynamic Window Approaches**

Basic Curvature Velocity Approach

- The curvature velocity approach enables the actual kinematic constraints and even some dynamic constraints of the robot to be taken into account during obstacle avoidance, which is an advantage over more primitive techniques.
- CVM begins by adding physical constraints from the robot and the environment to a velocity space.
- The velocity space consists of rotational velocity ω and translational velocity v , thus assuming that the robot travels only along arcs of circles with curvature

$$c = \omega/v.$$

Basic Curvature Velocity Approach

- Two types of constraints are identified:
 - those derived from the robot's limitations in acceleration and speed, typically $-\nu_{max} > \nu > \nu_{min}$ and $-\omega_{max} > \omega > \omega_{min}$
 - the constraints from obstacles blocking certain ν and ω values due to their positions.

The obstacles begin as objects in a Cartesian grid but are then transformed to the velocity space by calculating the distance from the robot position to the obstacle following some constant curvature robot trajectory.

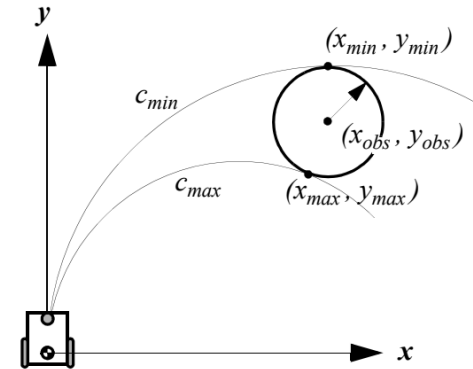


Figure 1: Tangent Curves from an obstacle.

Basic Curvature Velocity Approach

- To enable real-time performance, obstacles are simplified into circular shapes, and their contours are segmented into fewer intervals.
- The distance from the robot to the endpoints of these intervals is calculated, assuming a constant distance function between endpoints.
- A final velocity decision is determined by an objective function, evaluated within the velocity space adhering to kinematic, dynamic, and obstacle constraints.
- Utilizing a Cartesian grid for initial obstacle representation facilitates easy sensor fusion, especially in scenarios where a robot incorporates various ranging sensors.

Basic Curvature Velocity Approach

- CVM considers vehicle dynamics effectively but is limited by its circular simplification of obstacle shapes. While suitable for certain environments, this approach may lead to significant issues in others.
- Additionally, CVM is susceptible to local minima due to the absence of prior knowledge within the system.

Lane Curvature Method

- LCM calculates a set of desired lanes, trading off lane length and lane width to the closest obstacle.
- The lane with the best properties is chosen using an objective function. The local heading is chosen in such way that the robot will transition to the best lane if it is not in that lane already.
- Experimental results have demonstrated better performance as compared to CVM.
- One caveat is that the parameters in the objective function must be chosen carefully to optimize system behavior.

Obstacle Avoidance: Dynamic Window Approaches

- Another technique for taking into account robot kinematics constraints is the dynamic window obstacle avoidance method.
- A simple but very effective dynamic model gives this approach its name.
- Two such approaches are represented in the literature. One is local dynamic window approach and the other is global dynamic window approach.

1. Local Dynamic Window Approach

- In the local dynamic window approach, the kinematics of the robot is taken into account by searching a well-chosen velocity space.
- The velocity space is all possible sets of tuples (v, ω) where v is the velocity and ω is the angular velocity. The approach assumes that robots move only in circular arcs representing each such tuple, at least during one timestamp.
- Given the current robot speed, the algorithm first selects a dynamic window of all tuples (v, ω) that can be reached within the next sample period, taking into account the acceleration capabilities of the robot and the cycle time.

Obstacle Avoidance: Dynamic Window Approaches

1. Local Dynamic Window Approach

- The next step is to reduce the dynamic window by keeping only those tuples that ensure that the vehicle can come to a stop before hitting an obstacle. The remaining velocities are called admissible velocities.
- Note that the shape of the dynamic window is rectangular, which follows from the approximation that the dynamic capabilities for translation and rotation are independent.

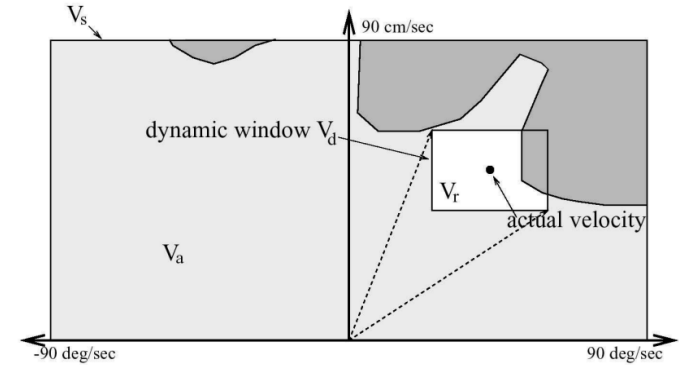


Figure 2: The dynamic window approach. The rectangular window shows the possible speeds (v, ω) and the overlap with obstacles in configuration space.

1. Local Dynamic Window Approach

- A new motion direction is chosen by applying an objective function to all the admissible velocity tuples in the dynamic window. The objective function prefers fast forward motion, maintenance of large distances to obstacles and alignment to the goal heading. The objective function O has the form

$$O = a.\text{heading}(\nu, \omega) + b.\text{velocity}(\nu, \omega) + c.\text{dist}(\nu, \omega)$$

- ❑ heading = Measure of progress toward the goal location;
- ❑ velocity = Forward velocity of the robot → encouraging fast movements
- ❑ dist = Distance to the closest obstacle in the trajectory

2. Global Dynamic Window Approach

- The global dynamic window approach integrates global thinking into the algorithm by incorporating NF1 (grassfire) into the objective function.
- NF1 assigns labels to cells in the occupancy grid based on the total distance to the goal.
- NF1 calculation is optimized by focusing on a selected rectangular region directed from the robot to the goal, adjusting the region's width if necessary to accommodate goal constraints.
- This approach offers some benefits of global path planning without requiring complete prior knowledge of the environment.

2. Global Dynamic Window Approach

- The occupancy grid is continually updated based on range measurements as the robot moves, enabling dynamic NF1 calculation.
- If NF1 calculation is impeded due to surrounding obstacles, the method reverts to the dynamic window approach to keep the robot in motion until a path becomes available.
- The global dynamic window approach promises real-time operation, dynamic constraint management, global planning, and effective obstacle avoidance even at high speeds.

2. Global Dynamic Window Approach

- Implementation with an omnidirectional robot using a 450 MHz on-board PC achieved a cycle frequency of approximately 15 Hz with a 5 cm resolution occupancy grid, sustaining average speeds exceeding 1 m/s during tests.
- There are also some popular Obstacle Avoidance approaches like
 1. The Schlegel approach
 2. Nearness diagram
 3. Gradient method
 4. Adding dynamic constraints

Mobile and Autonomous Robots

Obstacle Avoidance: Overview

Vector Field Histogram (VFH)			Bug			method	model fidelity
VFH* [322]	VFH+ [176, 323]	VFH [77]	Tangent Bug [161]	Bug2 [198, 199]	Bug1 [198, 199]		
circle	circle	simplistic	point	point	point	shape	
basic	basic					kinematics	
simplistic	simplistic					dynamics	
essentially local	local	local	local	local	local	view	other requisites
histogram grid	histogram grid	histogram grid	local tangent graph			local map	
						global map	
						path planner	
sonars	sonars	range	range	tactile	tactile	sensors	tested robots
nonholonomic (GuideCane)	nonholonomic (GuideCane)	synchro-drive (hexagonal)					
6 ... 242 ms	6 ms	27 ms				cycle time	performance
66 MHz, 486 PC	66 MHz, 486 PC	20 MHz, 386 AT				architecture	
fewer local minima	local minima	local minima, oscillating trajectories	efficient in many cases, robust	inefficient, robust	very inefficient, robust	remarks	

Mobile and Autonomous Robots

Obstacle Avoidance: Overview

Dynamic window		Curvature velocity		Bubble band		method	
Global dynamic window [81]	Dynamic window approach [130]	Lane curvature method [168]	Curvature velocity method [291]	Bubble band [165]	Elastic band [166]		
circle	circle	circle	circle	C-space	C-space	shape	model fidelity
(holonomic)	exact	exact	exact	exact		kinematics	
basic	basic	basic	basic			dynamics	
global	local	local	local	local	global	view	
	obstacle line field	histogram grid	histogram grid			local map	other requirements
C-space grid				polygonal	polygonal	global map	
NF1				required	required	path planner	
180° FOV SCK laser scanner	24 sonars ring, 56 infrared ring, stereo camera	24 sonars ring, 30° FOV laser	24 sonars ring, 30° FOV laser			sensors	
holonomic (circular)	synchro-drive (circular)	synchro-drive (circular)	synchro-drive (circular)	various	various	tested robots	
6.7 ms	250 ms	125 ms	125 ms			cycle time	performance
450 MHz, PC	486 PC	200 MHz, Pentium	66 MHz, 486 PC			architecture	
turning into corridors	local minima	local minima	local minima, turning into corridors			remarks	

Mobile and Autonomous Robots

Obstacle Avoidance: Overview

Other				method	model fidelity
Gradient method [171]	Global nearness diagram [225]	Nearness diagram [222, 223]	Schlegel [280]		
circle	circle (but general formulation)	circle (but general fomulation)	polygon	shape	
exact	(holonomic)	(holonomic)	exact	kinematics	
basic			basic	dynamics	
global	global	local	global	view	
	grid			local map	
local perceptual space	NF1		grid	global map	
fused			wavefront	path planner	
180° FOV distance sensor	180° FOV SCK laser scanner	180° FOV SCK laser scanner	360° FOV laser scanner	sensors	
nonholonomic (approx. circle)	holonomic (circular)	holonomic (circular)	synchrodrive (circular), tricycle (forklift)	tested robots	
100 ms (core algorithm: 10 ms)				cycle time	
266 MHz, Pentium				architecture	
		local minima	allows shape change	remarks	



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu



TA : Gunda Sukesh

Mobile and Autonomous Robots

UE23CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu



Mobile and Autonomous Robots

UE23CS343BB7

Unit-4: Navigation

Navigation Architectures

Mobile and Autonomous Robots

Navigation Architectures

- Integrating path planning, obstacle avoidance, localization, and perceptual interpretation into a complete robot system for real-world applications requires a systematic approach.
- One approach is to develop a custom-designed, application-specific software system that encompasses all functionalities for a specific purpose.
- While this may be efficient for simple mobile robot applications with limited features and short-term goals, it may not suffice for sophisticated and long-term systems.
- Addressing the mobility architecture issue involves designing navigation architectures, which are principled designs for the software modules comprising a mobile robot navigation system.

Mobile and Autonomous Robots

Navigation Architectures

- Navigation architectures aim to organize and integrate path planning, obstacle avoidance, localisation, and perception modules in a coherent and efficient manner.
- By adopting a principled approach to navigation architecture design, mobile robot systems can better handle the complexities of real-world environments and achieve long-term success.

Mobile and Autonomous Robots

Navigation Architectures

Modularity for code reuse and sharing

- Basic software engineering principles embrace software modularity, and the same general motivations apply equally to mobile robot applications.
- But modularity is of even greater importance in mobile robotics because in the course of a single project the mobile robot hardware or its physical environmental characteristics can change dramatically, a challenge most traditional computers do not face.
- For example, one may introduce a Sick laser range- finder to a robot that previously used only ultrasonic rangefinders. Or one may test an existing navigator robot in a new environment where there are obstacles that its sensors cannot detect, thereby demanding a new path-planning representation.

Mobile and Autonomous Robots

Navigation Architectures

- The goal is to modify parts of the robot's functionality without causing unintended side effects that require revisiting other aspects of its operation.
- For example, retaining the obstacle avoidance module intact despite changes in sensor suite or kinematic structure, such as transitioning from a tricycle chassis to a differential drive chassis, is ideal.
- Maintaining modularity allows for easier adaptation and evolution of mobile robot systems to varying hardware configurations and environmental conditions.

Mobile and Autonomous Robots

Navigation Architectures

Control Localisation

- Localisation of robot control is an even more critical issue in mobile robot navigation.
- At the other extreme, high-level planning and task decision making are required for robots to perform useful roles in their environment.
- For example, localising obstacle avoidance software ensures stability and focused verification of its implementation, while high-level planning software can be localized for exhaustive testing in simulation.
- It is also valuable to localise such high-level decision-making software, enabling it to be tested exhaustively in simulation and thus verified even without a direct connection to the physical robot.

Mobile and Autonomous Robots

Navigation Architectures

- Localisation of control can enable a specific learning algorithm to be applied to just one aspect of a mobile robot's overall control system.
- Such targeted learning is likely to be the first strategy that yields successful integration of learning and traditional mobile robotics.
- The advantages of localisation and modularity provide a compelling case for the use of principled navigation architectures.

Mobile and Autonomous Robots

Navigation Architectures

Decompositions

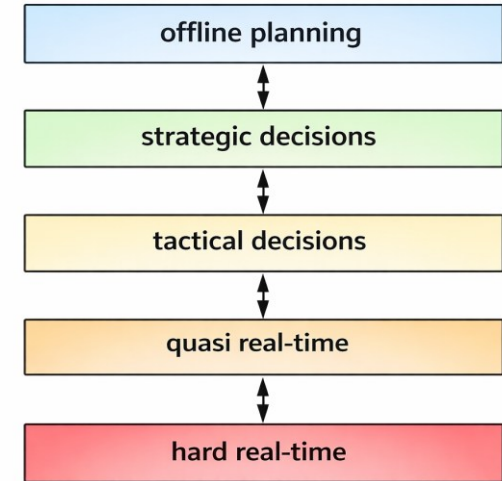
- Decompositions identify axes along which we can justify discrimination of robot software into distinct modules.
- Decompositions also serve as a way to classify various mobile robots into a more quantitative taxonomy. It is mainly classified into two types.
 1. **Temporal decomposition** distinguishes between real-time and non real-time demands on mobile robot operation.
 2. **Control decomposition** identifies the way in which various control outputs within the mobile robot architecture combine to yield the mobile robot's physical actions.

Mobile and Autonomous Robots

Navigation Architectures

1. Temporal Decomposition

- A temporal decomposition of robot software distinguishes between processes that have varying real-time and non-real-time demands.
- The figure depicts a generic temporal decomposition for navigation.
- In this figure, the most real-time processes are shown at the bottom of the stack, with the highest category being occupied by processes with no real time demands.
- In this example, the lowest level ensures a fast cycle time, like a 40 Hz bandwidth.



Mobile and Autonomous Robots

Navigation Architectures

1. Temporal Decomposition (contd.)

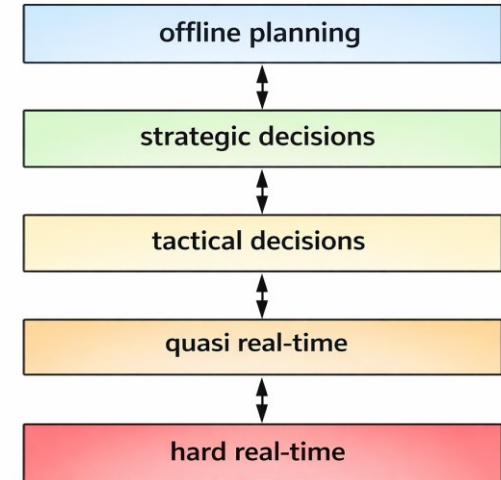
- A quasi real-time layer may require a 0.1 second response time with large allowable worst-case cycle times.
- A tactical layer involves decision-making affecting immediate actions, subject to temporal constraints.
- A strategic or offline layer influences long-term behavior with fewer temporal constraints.
- Four important, interrelated trends correlate with temporal decomposition. They are sensor response time, temporal depth, spatial locality and context specificity.

Mobile and Autonomous Robots

Navigation Architectures

a. Sensor Response Time

- A particular module sensor response time can be defined as the amount of time between acquisition of a sensor event and a corresponding change in the output of the module.
- As one moves up the stack in figure, the sensor response time tends to increase. For the lowest-level modules, the sensor response time is often limited only by the raw processor and sensor speeds.
- At the highest-level modules, sensor response can be limited by slow and deliberate decision-making processes.



b. Temporal Depth

- Temporal depth is a useful concept applying to the temporal window that affects the module's output, both backward and forward in time.
- Temporal horizon describes the amount of look ahead used by the module during the process of choosing an output.
- Temporal memory describes the historical time span of sensor input that is used by the module to determine the next output.
- Lowest-level modules tend to have very little temporal depth in both directions, whereas the deliberative processes of highest-level modules make use of a large temporal memory and consider actions based on their long-term consequences, making note of large temporal horizons.

Mobile and Autonomous Robots

Navigation Architectures

c. Spatial Locality

- Hand in hand with temporal span, the spatial impact of layers increases dramatically as one moves from low-level modules to high-level modules.
- Real-time modules tend to control wheel speed and orientation, controlling spatially localized behavior.
- High-level strategic decision making has little or no bearing on local position, but it informs global position far into the future.

d. Context Specificity

- A module makes decisions as a function not only of its immediate inputs but also as a function of the robot's context as captured by other variables, such as the robot's representation of the environment.
- Lowest-level modules tend to produce outputs directly as a result of immediate sensor inputs, using little context and therefore being relatively context insensitive.
- Highest-level modules tend to exhibit very high context specificity.
- For strategic decision-making, given the same sensor values, dramatically different outputs are nevertheless conceivable depending on other contextual parameters.

Mobile and Autonomous Robots

Navigation Architectures

d. Context Specificity (contd.)

- An example demonstrating these trends is depicted in figure, which shows a temporal decomposition of a simplistic navigation architecture into four modules.
- At the lowest level the PID control loop provides feedback to control motor speeds.
- An emergency stop module uses short-range optical sensors and bumpers to cut current to the motors when it predicts an imminent collision.
- Knowledge of robot dynamics means that this module by nature has a greater temporal horizon than the PID module.

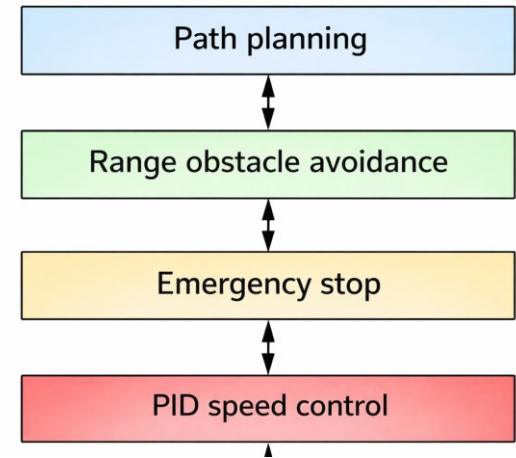


Figure: A four-level simple Temporal Decomposition

Mobile and Autonomous Robots

Navigation Architectures

d. Context Specificity (contd.)

- The next module uses longer range laser range finding sensor returns to identify obstacles well ahead of the robot and make minor course deviations.
- Finally, the path planner module takes the robot's initial and goal positions and produces an initial trajectory for execution, subject to change based on actual obstacles that the robot collects along the way.
- The cycle time, or bandwidth, of the modules changes by orders of magnitude between adjacent modules.
- Such dramatic differences are common in real navigation architectures, and so temporal decomposition tends to capture a significant axis of variation in a mobile robot navigation architecture.

Mobile and Autonomous Robots

Navigation Architectures

2. Control Decomposition

- Control decomposition identifies the way in which each module's output contributes to the overall robot control outputs.
- Presentation of control decomposition requires the evaluator to understand the basic principles of discrete systems representation and analysis.
- Modules form a closed system, with each having one output and one or more inputs.
- The physical robot and environment are represented as special modules, simplifying inputs and outputs.
- Control pathways range from pure sequential to fully parallel architectures. Sequential systems ensure predictability and verifiability but may lack flexibility.

Mobile and Autonomous Robots

Navigation Architectures

2. Control Decomposition (contd.)

- Fully parallel systems distribute control across multiple modules.
- Switched parallel control attributes output to one module at a time, useful for clear decision-making.
- Mixed parallel control combines outputs from multiple modules mathematically, offering flexibility but requiring careful implementation.
- Both parallel architectures are common in behavior robotics, each with advantages and challenges.
- Verification of parallel control systems can be difficult due to the complexity of interactions.

Mobile and Autonomous Robots

Navigation Architectures

2. Control Decomposition (contd.)

- Parallel control mimics biological strategies, aiming to leverage the advantages of true parallelism in complex organisms.
- Control decomposition discriminates between different types of control pathways through the portion of this system comprising the robot algorithm.
- There are two types of these control pathways
 - a. Serial/Linear
 - b. Parallel.

Mobile and Autonomous Robots

Navigation Architectures

a. Serial Decomposition

- Serial control decomposition involves a linear or sequential control pathway within the system, where modules are connected in a linear manner, allowing for relatively straightforward verification of robot performance.
- It uses the internal state of all associated modules and the value of the robot's percept in a sequential manner to compute the next robot action.

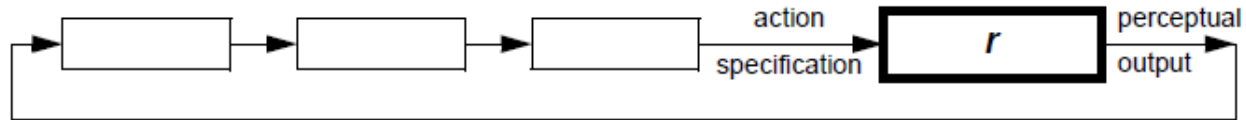


Figure 6.25
Example of a pure serial decomposition.

Mobile and Autonomous Robots

Navigation Architectures

a. Serial Decomposition (contd.)

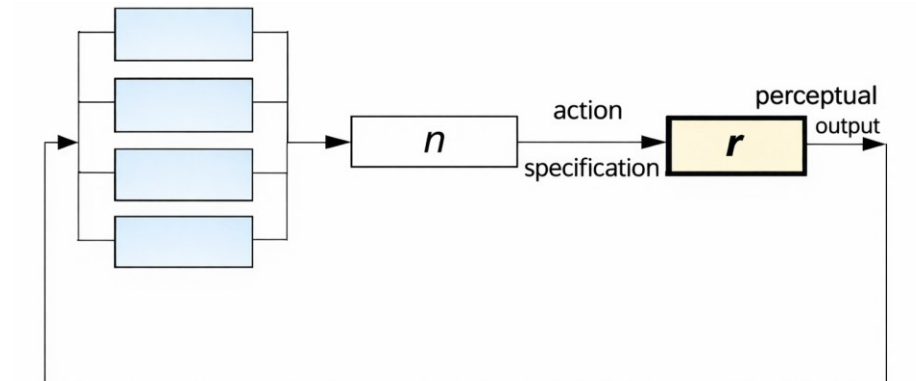
- A pure serial architecture has better verifiability and predictability. Since the state and outputs of each module depend entirely on the inputs it receives from the module upstream, the entire system, including the robot, is a single well-formed loop.
- The control flow is a linear sequence through a string of modules.
- Therefore, the overall behaviour of the system can be evaluated using discrete forward simulation methods.

Mobile and Autonomous Robots

Navigation Architectures

b. Parallel Decomposition

- The fully parallel system distributes responsibility for the system's control output across multiple modules.
- Involves a more complex interconnected control pathway within the system, making verification of robot performance more challenging due to the parallel nature of connections and interactions between modules.
- There are 2 common parallel architectures; mixed and switched.



Mobile and Autonomous Robots

Navigation Architectures

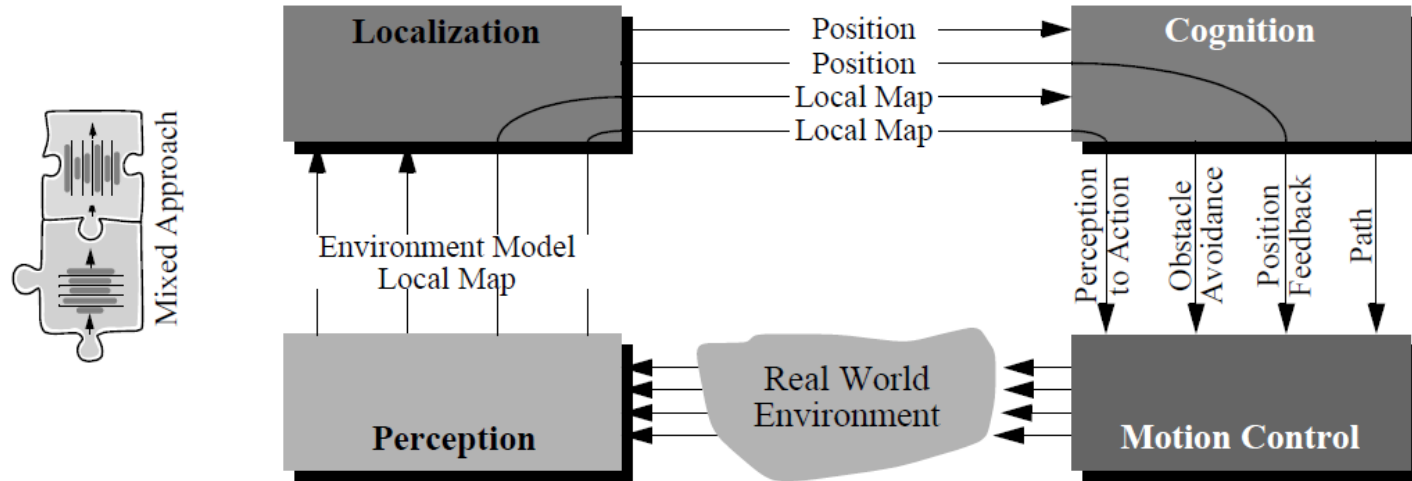
b. Parallel Decomposition (contd.)

- An important advantage of parallel control is its biomimetic aspect that complex organic organisms benefit from through large degrees of true parallelism (e.g., the human eye).
- However, in parallel control, verification of robot performance can be extremely difficult.
- Because such systems include truly parallel, multithreaded implementations, the intricacies of robot-environment interaction and sensor timing required to properly represent all conceivable module-module interactions can be difficult to simulate.
- So, much testing in the parallel control community is performed empirically using physical robots.

Mobile and Autonomous Robots

Navigation Architectures

Case study: tiered robot architectures



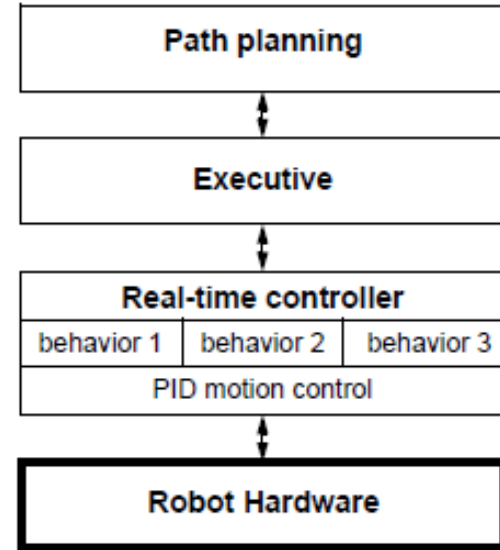
Mobile and Autonomous Robots

Navigation Architectures

Case study: tiered robot architectures

Temporal Decomposition

- **Path Planning** (Strategic Layer):
 - Non-real-time.
 - Generates optimal global paths using complete map info.
- **Executive** (Tactical Layer):
 - Bridges planning and real-time execution.
 - Activates behaviors, handles failures, manages short-term memory.
- **Real-Time Controller** (Reactive Layer):
 - Implements behaviors (e.g., obstacle avoidance, wall-following).
 - Runs PID loops for motor control.
- **Robot Hardware**:
 - Executes motor commands, reads low-level sensors.



A general tiered mobile robot navigation architecture based on a temporal decomposition



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu





Mobile and Autonomous Robots

UE23CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu

TA : Gunda Sukesh



Mobile and Autonomous Robots

UE23CS343BB7

Unit- 4: Navigation

**Basics of Reinforcement Learning using,
Applications and Social Implications**

Mobile and Autonomous Robots

Reinforcement Learning

- Navigation serves as the fundamental pillar of robotic autonomy, enabling agents to traverse complex environments.
- While traditional frameworks rely on static maps, sensor fusion, and deterministic algorithms, these methods often struggle with the uncertainty of real-world scenarios.
- Consequently, modern research has shifted toward data-driven paradigms.
- Specifically, **Reinforcement Learning (RL)** allows robots to optimize navigation policies through environmental interaction, while **Imitation Learning (IL)** streamlines the process by training models to replicate expert human demonstrations.

Mobile and Autonomous Robots

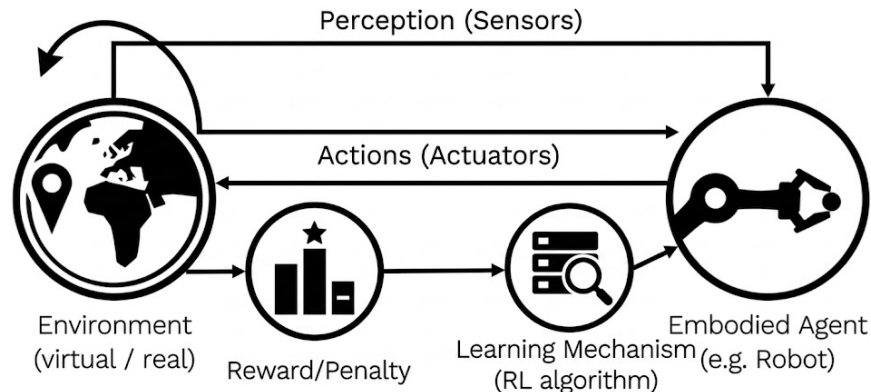
Reinforcement Learning



Mobile and Autonomous Robots

Reinforcement Learning

- Reinforcement learning (RL) offers a powerful approach for training robots to perform tasks through trial and error.
- Unlike traditional programming where robots are explicitly instructed on every step, RL allows robots to learn from their interactions with the environment.
- This makes RL particularly well-suited for complex tasks or dynamic environments where pre-programming every possible scenario might be impractical.



Mobile and Autonomous Robots

Reinforcement Learning

Core RL Terminologies for Robotics

- **Agent:** The autonomous robot (or controller) that perceives data and makes decisions.
- **Environment:** The workspace either a physical arena or a digital twin, where the robot operates and receives feedback.
- **Action (A):** The discrete or continuous control signals (e.g., wheel velocities, joint torques) executed by the robot.
- **State (S):** A comprehensive snapshot of the robot's situation, integrating proprioceptive data (encoder/IMU) and exteroceptive data (LiDAR/Camera).
- **Reward (R_t):** A scalar feedback signal used to optimize the policy(π); it defines the goal by penalizing collisions and rewarding progress toward a target.

Reinforcement Learning: The Learning Process

Steps	Definition	Purpose	Robotic Example
Exploration	The robot interacts with its environment by taking various actions to discover their effects.	To gather information about the environment, especially when the outcomes of actions are unknown.	A robotic arm trying different grip strengths to pick up objects of varying weights.
Reward Feedback	A scalar signal (R_t) quantifying success of a state transition.	To guide the robot toward favorable behaviors by reinforcing successful actions.	A robot receives a positive reward for successfully navigating to a target location.
Learning and Adaptation	Updating Policy (π) to maximise long-term return.	To improve decision-making over time, enabling the robot to choose actions that maximise cumulative rewards.	Adjusting movement patterns to enhance balance and speed in a walking robot.
Exploitation	Selecting actions with highest estimated value from knowledge.	To efficiently achieve goals by utilising accumulated knowledge.	A delivery robot consistently choosing the fastest route it has learned to reach destinations.

Mobile and Autonomous Robots

Reinforcement Learning

The Exploration-Exploitation Trade-off

Effective RL hinges on the balance between "searching" (Exploration) and "earning" (Exploitation).

- **ϵ -Greedy:** Occasionally takes a random "risk" to find better paths, otherwise stays with the known best.
- **UCB (Upper Confidence Bound):** Prioritizes actions with high uncertainty, ensuring no part of the environment is left un-probed.

Mobile and Autonomous Robots

Reinforcement Learning

Real World Example

Researchers at MIT trained a quadruped robot, Mini Cheetah, using reinforcement learning. The robot learned to run at high speeds by exploring various movement patterns in simulation, receiving rewards for faster locomotion. The learned policy was then transferred to the physical robot, enabling it to adapt to different terrains without explicit programming.



Mobile and Autonomous Robots

Reinforcement Learning

Challenges(Mini Cheetah)

1. **Sample Inefficiency:** RL algorithms often require a vast number of interactions with the environment, which is impractical for real-world robotic applications due to time and resource constraints.
2. **Exploration vs. Exploitation:** Striking a balance between exploring new actions and exploiting learned successful actions can be difficult.
3. **Reward Design:** Designing appropriate reward functions is challenging; poorly designed rewards can lead to unintended behaviors or hinder learning.
4. **Computational Cost:** RL algorithms can be computationally expensive, especially for complex tasks with high-dimensional state spaces.

Mobile and Autonomous Robots

Reinforcement Learning

Challenges(Mini Cheetah)

5. **Safety Concerns:** During training, robots may perform unsafe actions, posing risks to themselves and their environment. Ensuring safe exploration is a significant challenge.
6. **Multi-Agent Coordination:** Coordinating multiple robots introduces complexity due to the need for communication and synchronization among agents.

Mobile and Autonomous Robots

Reinforcement Learning: Applications

1. Industrial Robotics

- a. **Assembly and Manipulation:** Robots can learn intricate assembly tasks by interacting with objects and receiving rewards for successful manipulations.
- b. **Quality Control:** RL agents can visually inspect products and identify defects, adapting to variations in product appearance.
- c. **Logistics and Warehousing:** Robots can navigate warehouses, pick and place objects efficiently, and optimize path planning based on dynamic environments.



2. Service Robotics

- a. **Domestic Robots:** RL can enable robots to learn household chores like cleaning and tidying up, adapting to different environments and user preferences.
- b. **Eldercare Assistance:** Robots can learn to provide personalized assistance to elderly individuals, such as medication reminders and object retrieval.
- c. **Search and Rescue:** Robots can navigate disaster zones and locate survivors, adapting to unstructured and potentially dangerous environments.



3. Robotics for Healthcare

- a. **Surgical Robots:** RL can assist surgeons by controlling robotic arms during minimally invasive procedures and learning optimal surgical maneuvers.
- b. **Physical Rehabilitation:** Robots can personalise rehabilitation exercises for patients based on their progress and feedback.



4. Autonomous Vehicles

- a. **Self Driving Cars:** RL is a key technology for enabling self-driving cars to navigate complex road environments, learn traffic rules, and adapt to unpredictable situations.
- b. **Delivery Drones:** RL can help drones plan efficient routes for deliveries, avoiding obstacles and optimizing flight paths based on weather conditions.



5. Legged Robots

- a. **Locomotion Control:** RL can enable legged robots to learn various gaits, navigate uneven terrain, and adapt to challenging environments.
- b. **Humanoid Robots:** RL can help humanoid robots learn complex motor skills like walking, running, and manipulating objects in a human-like manner.



- **Job Displacement:** Automation powered by RL could potentially displace human workers in various industries, leading to unemployment and economic strain.
- **Safety Concerns:** Failures or malfunctions could pose safety risks, particularly in like self-driving cars or medical robots.
- **Ethical Considerations:** Designing reward functions for RL algorithms raises ethical questions. Who defines what constitutes "good" behavior for robots, and how do we ensure they align with human values?
- **Bias and Fairness:** RL algorithms trained on biased data could perpetuate societal inequalities. It's crucial to ensure fairness and inclusivity in the development and deployment of RL-powered robots.
- **Transparency and Explainability:** Understanding how RL algorithms reach decisions is crucial for ensuring accountability and trust in their applications.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu